Web-Based Project Management Application for ProManage Solutions

u5602780

20th May 2025

Contents

1 Introduction

This report presents the development of a secure web-based project management application for ProManage Solutions, designed to enhance team collaboration and productivity. Built using the Flask Python framework, the application incorporates secure authentication, role-based access control (RBAC), password security, and comprehensive activity logging. This document details the application's architecture, implemented features, design and security decisions, additional functionalities, and development process, addressing the module learning outcomes.

2 Application Architecture

The application leverages Flask, a lightweight Python web framework, to provide a modular and scalable structure. It integrates user management, project management, and robust security features, using JSON files for data persistence. The architecture is designed as a tree-like structure, with the dashboard serving as the central hub branching into projects, and tasks branching from each project. A login screen acts as the entry point, ensuring all access is authenticated.

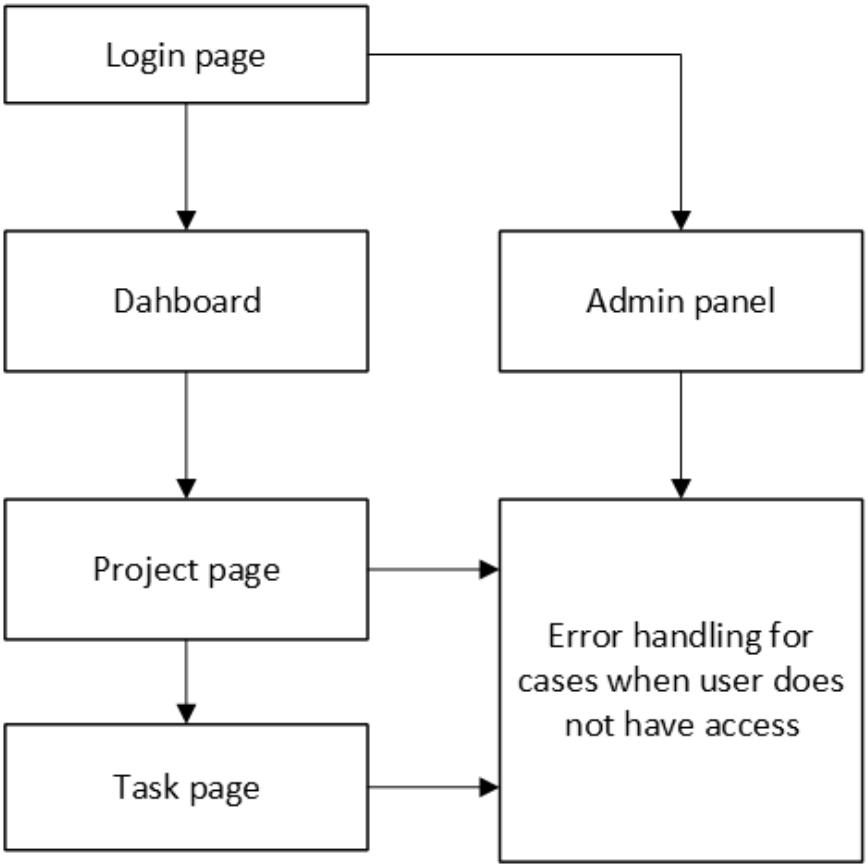

Figure 1: Architecture diagram of the web-based project management application

2.1 System Components

The Flask application is organised into distinct modules and routes:

- **User Manager (UM)**: Defined in `user_manager.py`, this module handles user authentication, roles, and credentials, interfacing with `user_credentials.json`.
- **Project Management System (PM)**: Implemented in `task_manager.py`, it manages project and task operations, using `projects.json` for data storage and `access_config.json` for role-based permissions.
- **JWT Manager**: Located in `jwt_manager.py`, it manages JSON Web Token (JWT) creation and validation for secure session handling.

JWT tokens are stored in HTTP-only cookies with a 3600-second lifespan, ensuring secure user sessions.

2.2 Database Schema

Data is stored in JSON files, providing a lightweight solution suitable for managing a few hundred projects:

- **user_credentials.json**: Stores usernames, hashed passwords, and roles.
- **projects.json**: Contains project details, including titles, descriptions, tasks, assigned users, and activity logs.
- **access_config.json**: Defines role-based permissions for operations.

While sufficient for the application's scope, a relational database could enhance scalability for larger deployments.

2.3 Security Architecture

Security is a cornerstone of the application:

- **Authentication**: JWT tokens are issued upon login and validated for every action, redirecting users to the login page if tokens are invalid or expired, preventing unauthorised access.
- **Authorisation**: RBAC, configured via `access_config.json`, restricts actions based on user roles, ensuring robust access control.
- **Password Security**: Passwords are salted and hashed using the bcrypt library, offering superior protection compared to simpler algorithms like SHA256.
- **Logging**: All actions are logged to `app.log`, with warnings for suspicious activities (e.g., failed login attempts), supporting auditing and debugging.

3 Implemented Features

The application provides tailored functionalities for team members, project managers, and administrators, ensuring efficient project and user management.

3.1 Team Member Operations

- **Login**: Users authenticate via the `/login` route, receiving a JWT token upon successful validation.
- **Task Management**: Team members can view their assigned tasks and update statuses (e.g., "in progress" or "complete") using routes like `/project/<projectId>/task/<taskI` and `/mark_complete`.
- **Project Progress**: The `/project/<projectId>` route displays project details and tasks assigned to the user, with a table summarising task assignments and activity logs.

3.2 Project Manager Operations

- **Dashboard**: Accessible via `/dashboard`, it provides an overview of project statuses and tasks for non-admin users.
- **Task Creation and Assignment**: Managers can create tasks (`/create/<projectId>/task`) and assign users (`/project/<projectId>/task/<taskId>/assign_user_to_task`).
- **Team Management**: Managers can add or remove users from projects (`/project/<projectId>/ /remove_user_from_project`).
- **Activity Logs**: Comprehensive logs of all project and task activities are available in `app.log`, with project pages displaying task assignment summaries.

3.3 Administrative Operations

- **User Management**: Admins access the `/admin` route to create (`/admin/create_user`), delete (`/admin/delete_user`), assign passwords (`/admin/assign_password`), and set roles (`/admin/set_role`). The admin account is restricted to the admin panel for security, preventing project management activities.

3.4 Preconfigured Users for Testing

For testing purposes, the application includes preconfigured users:

- **Admin**: Username: admin, Password: admin
- **User (Password)**: Alice (a), Bob (b), Charlie (c), Dave (d), Eve (e)

Each user has associated projects and tasks, facilitating evaluation.

4 Design and Security Decisions

The application's design prioritises usability, security, and maintainability, with decisions driven by the need to protect against common vulnerabilities.

4.1 Authentication Mechanism

JWT was chosen for its stateless, secure authentication model, implemented via `jwt_manager.py`. Tokens are stored in HTTP-only cookies with a 3600-second lifespan, balancing security and user convenience. Passwords are salted and hashed using bcrypt, which provides robust protection against brute-force attacks compared to weaker algorithms like SHA256.

4.2 Role-Based Access Control

RBAC, configured through `access_config.json`, ensures that only authorised users perform specific actions. For example, routes like `/create/project` and `/delete/task` validate roles via `ProjectManagementSystem`, preventing unauthorised access. The admin account's restriction to the admin panel reduces the risk of over-privileged account misuse.

4.3 Logging

The logging system, configured with `logging.basicConfig`, records all user actions in `app.log`, including timestamps and usernames. Suspicious activities, such as failed logins, trigger warnings, enabling proactive security monitoring and debugging.

5 Additional Functionality

Beyond core requirements, the application includes:

- **Admin Panel**: The `/admin` route offers a comprehensive interface for user management, enhancing administrative control and security.
- **Task Status Updates**: Team members can mark tasks as "in progress" or "complete," improving project tracking and visibility.
- **User-Friendly Interface**: The UI/UX design prioritises intuitiveness, with features like closing pop-ups using the "Esc" key, alongside "Cancel" and "Close" buttons, enhancing user experience.

6 Development Process

The application was developed iteratively, focusing on incremental functionality and robust security measures.

6.1 Development Steps

1. **Flask Initialisation**: Established the Flask app with core routes for login (`/login`) and dashboard (`/dashboard`).

2. **Module Implementation**: Developed `UserManager` for user operations and `ProjectManagement` for project and task management.

3. **Frontend Development**: Iterated through three UI versions: pure HTML, HTML with CSS, and finally Tailwind CSS, which simplified code, improved portability, and enhanced readability.

4. **Security Integration**: Implemented JWT authentication, RBAC, and logging to secure the application.

5. **Testing**: Used Flask's debug mode and log analysis to test functionality, attempting to exploit vulnerabilities to ensure robustness.

6.2 Debugging and Challenges

- **Token Expiration**: An initial issue with the `datetime` library caused incorrect token expiration times. This was resolved by updating to the correct library and handling exceptions in `decode_jwt`, redirecting users to the login page upon token expiration.

- **Permission Enforcement**: Early iterations lacked role checks on some routes, risking unauthorised actions like task deletion. This was addressed by adding comprehensive role validation across all routes, ensuring secure access control.

References

[1] Flask Documentation, (2025). Available at: `https://flask.palletsprojects.com/` [Accessed 20 May 2025].

[2] JSON Web Tokens, (2025). Available at: `https://jwt.io/` [Accessed 20 May 2025].

[3] bcrypt Documentation, (2025). Available at: `https://pypi.org/project/bcrypt/` [Accessed 20 May 2025].

A Source Code

The complete source code is included in the submission as a zip file containing `main.py` and supporting modules.